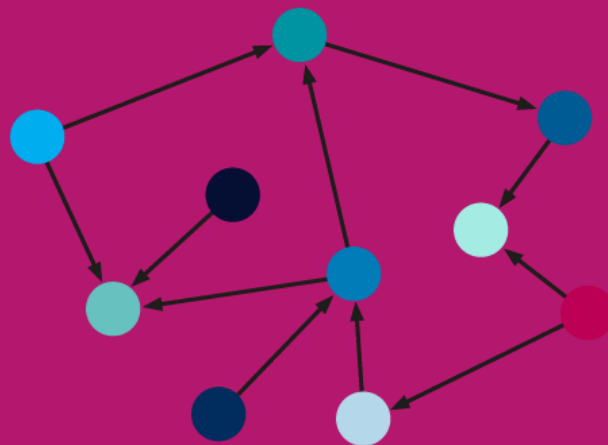




Cassandra

Жишкович Андрій Володимирович

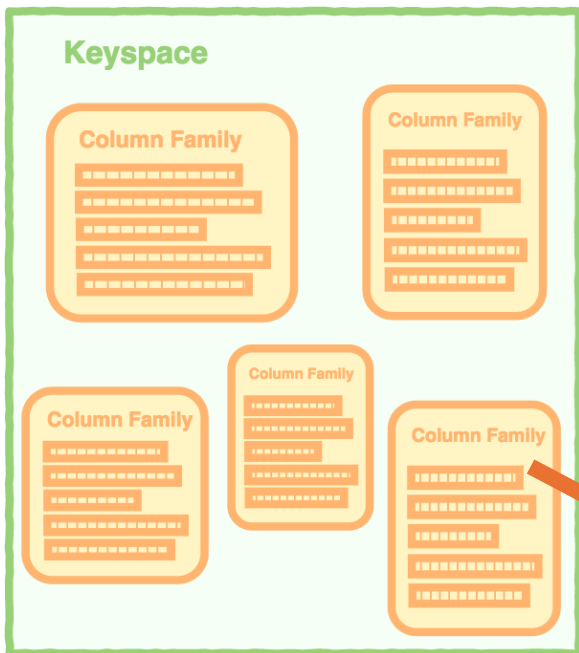
Ширококолонкова база даних

Ширококолонкова база даних — це тип NoSQL баз даних яка використовує колонково-орієнтовану модель.

У таких базах даних кожен рядок може мати різну кількість стовпців, що дозволяє ефективно зберігати та обробляти великі обсяги неоднорідних даних.

Вони оптимізовані для швидкого запису та читання, підтримують горизонтальне масштабування та забезпечують високу доступність і відмовостійкість у розподілених системах.

Apache Cassandra та Apache HBase є двома найвідомішими прикладами.



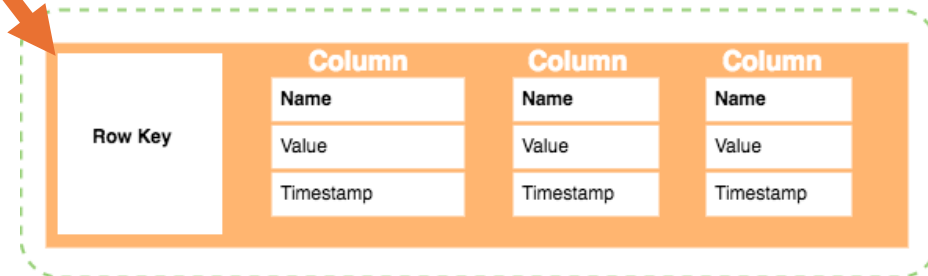
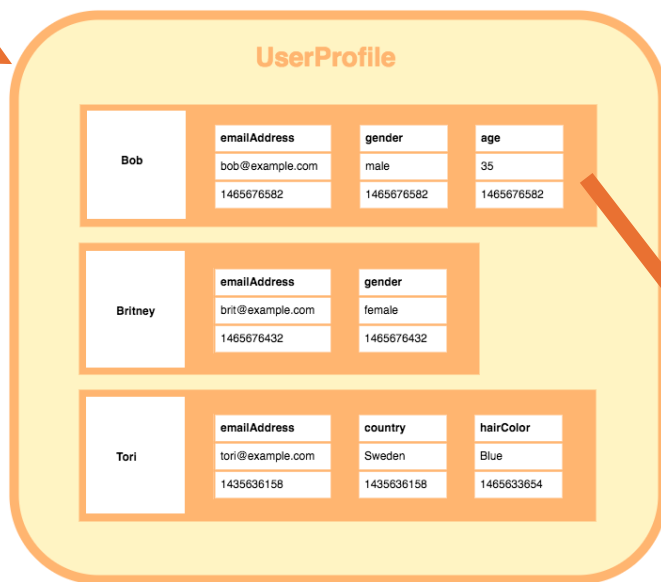
ключовий простір →

сімейство колонок →

рядки →

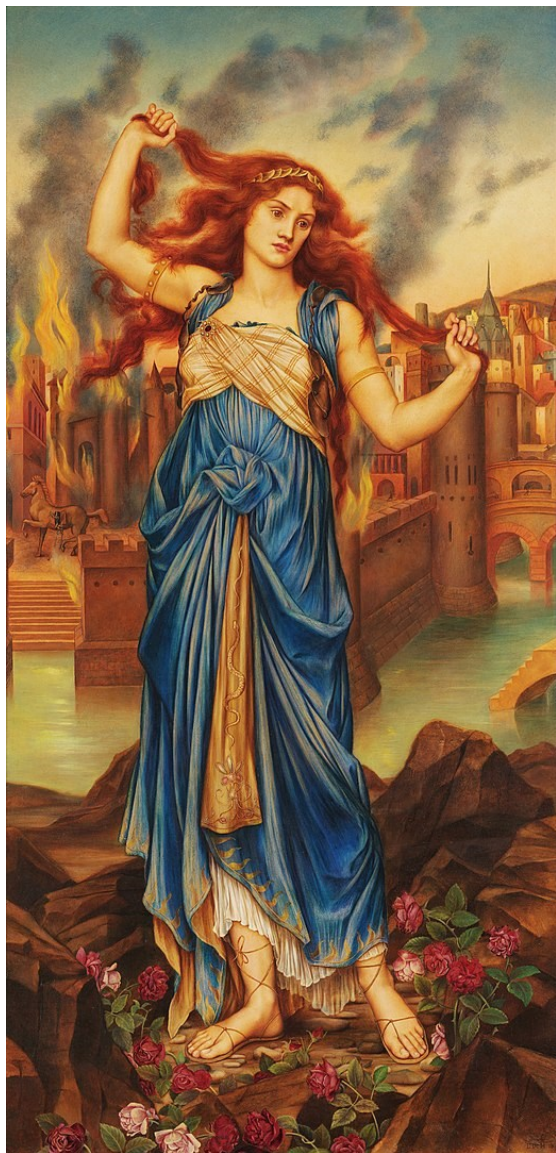
колонки →

пари ключ/значення + часова мітка



Cassandra

Що таке Apache Cassandra



Apache Cassandra — це ширококолонкова, консистентна, розподілена система управління базами даних, що належить до класу NoSQL-систем.

Розрахована на створення високомасштабованих і надійних сховищ великих масивів даних, представлених у вигляді хеша.

Була розроблена для роботи на недорогому стандартному обладнанні та здатна обробляти високий потік записів без втрати ефективності читання.

Розробники СУБД назвали свій проєкт на честь міфологічної троянської пророчиці Кассандри. Це була алюзія на прокляття оракула. Нагадаємо собі, що Oracle Corporation спеціалізується на випуску систем керування БД.

Назва "Cassandra" походить від персонажа давньогрецької міфології. Кассандра була дочкою троянського царя Пріама і цариці Гекуби, відомою за свою здатність передбачати майбутнє. Однак, за легендою, вона була проклята богом Аполлоном так, що ніхто не вірив її пророцтвам



[Youtube: Cassandra in 100 Seconds](#)

Історія та розвиток Apache Cassandra



Історично **Cassandra** базувалася на **Google Bigtable** та **Amazon DynamoDB**. Вона взяла розподілену природу Dynamo і побудувала свою модель даних, використовуючи Bigtable, поєднавши їх у високодоступне сховище даних з багатою моделлю даних.

Apache Cassandra була розроблена в Facebook у 2008 році для покращення пошуку вхідних повідомлень та вирішення проблем масштабованості і продуктивності їхньої інфраструктури.

Історія та розвиток Apache Cassandra



Вона була створена як масивна, глобально розподілена база даних з високою доступністю, здатна масштабуватися до розмірів сучасних вебдодатків та соціальних мереж.

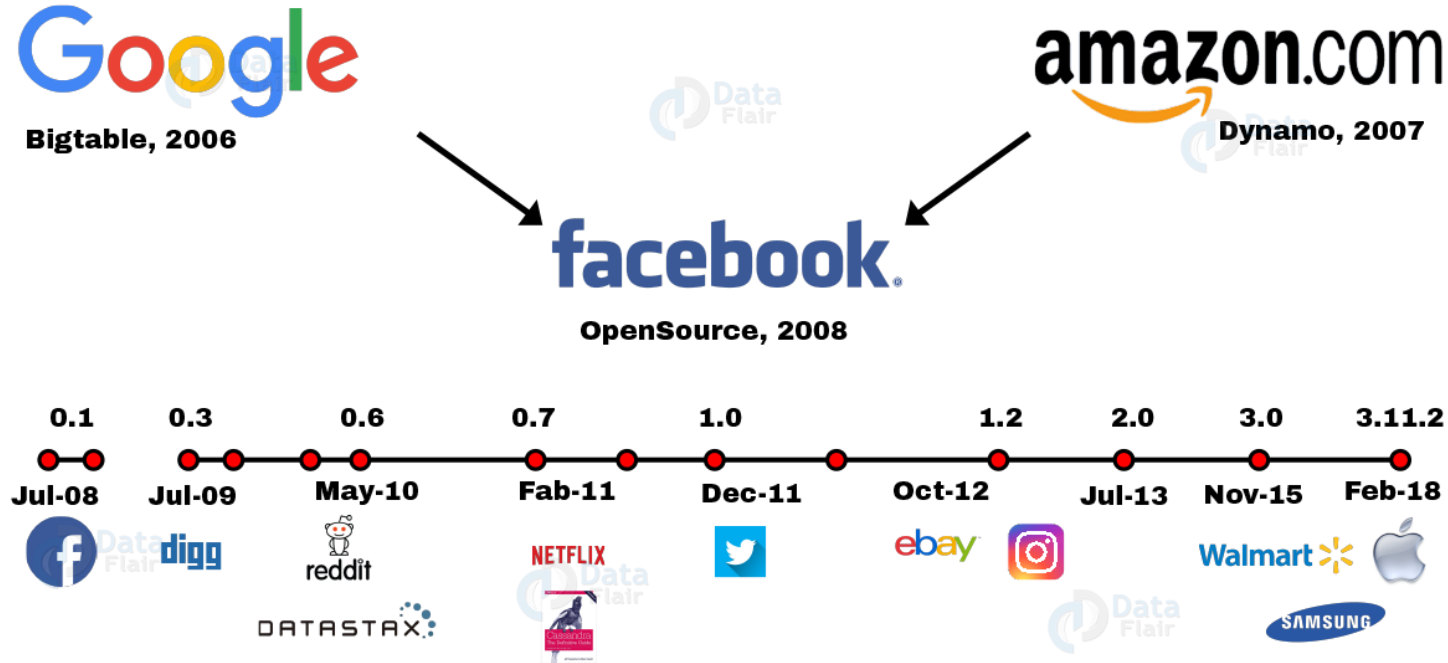
У 2009 році Cassandra була передана спільноті як проект з відкритим кодом під егідою Apache Software Foundation.

Вона здобула справжню популярність, коли у 2009-2010 роках стала проектом інкубатора верхнього рівня Apache.

Протягом кількох років Apple, Netflix, Instagram та інші глобальні компанії, які потребують високої продуктивності для великих обсягів даних, долучилися до проекту.

Історія та розвиток Apache Cassandra

Згодом Meta замінила Cassandra на HBase для певних проєктів, але продовжує використовувати Cassandra у своєму іншому проєкті — Instagram, який обслуговує понад 1 мільярд активних користувачів щомісяця.



З часом Cassandra стала популярною завдяки своїй здатності обробляти великі обсяги даних у розподілених системах.

Вона використовується такими компаніями, як Apple, Netflix, Twitter, Reddit, Cisco, IBM, Cloudkick, Digg, Rackspace, Instagram та інших, для різноманітних завдань від потокового передавання даних до управління великими обсягами користувацької інформації.

Apache Cassandra в Netflix



- 98% потокових даних зберігається в Apache Cassandra
- Дані включають деталі клієнтів, історію переглядів, виставлення рахунків та платежі
- Основне сховище даних для обслуговування мільйонів операцій на секунду
- Вони мають сотні кластерів
- Обробляють **300 мільйонів операцій в секунду (OPS)** на найбільш активних окремих кластерах
- **500 ТБ** – найбільш щільний окремий кластер
- Кластери розподілені по всьому світу з використанням кількох дата-центрів.

Netflix зробив значний внесок в інструментарій екосистеми і був дуже активним, разом з іншими учасниками, такими як Instagram, і це навіть не найбільших масштаб.

Apache Cassandra в Apple



- Понад **160 000** екземплярів Apache Cassandra
- Збережено понад **100 ПБ**
- Кілька мільйонів операцій в секунду (OPS)
- Тисячі кластерів

Як ви бачите тут, масштаб Apple значно вищий.

Apache Cassandra

Є вісім функцій, які роблять Cassandra потужною:

1. Готовність до роботи з великими даними
2. Продуктивність читання/запису
3. Лінійна масштабованість
4. Найвища доступність
5. Самовідновлення та автоматизація
6. Географічний розподіл
7. Незалежність від платформи
8. Незалежність від постачальників

Готовність до роботи з великими даними

Big Data Ready

- Partitioning over distributed architecture makes the database capable of handling data of any size, up to and including petabytes
- Need more volume? Add more nodes

Розподіл по розподіленій архітектурі робить базу даних здатною обробляти дані будь-якого розміру: на рівні петабайтів.

Потрібно більше обсягу? Додайте більше вузлів.

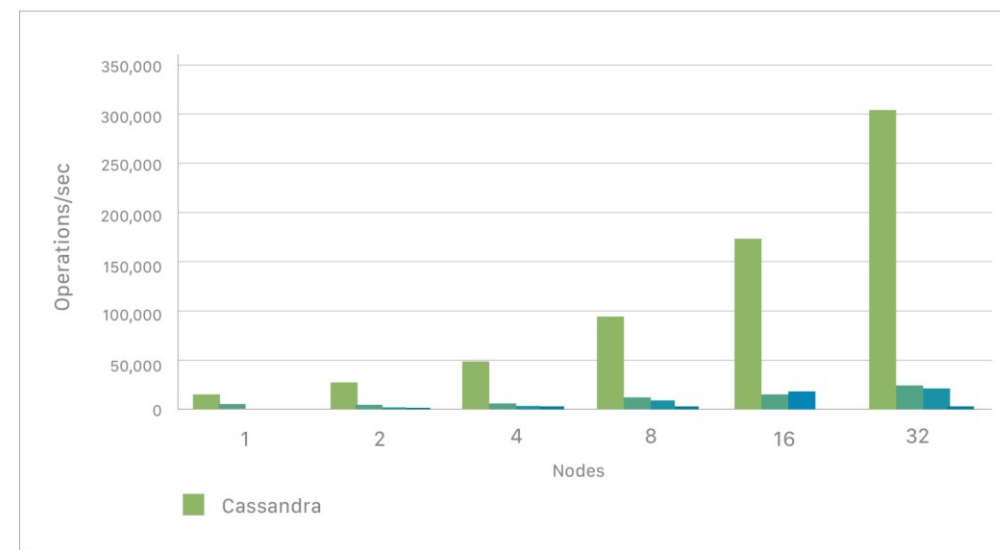


Продуктивність читання/запису

Read/Write Performance

- Even a single Cassandra node is very performant, but a cluster consisting of multiple nodes and datacenters brings throughput to the next level
- Decentralization, where there are no leader or follower nodes, means that every node is able to deal with any request, read or write

Balanced Read/Write Mix



Один вузол дуже продуктивний, але кластер з кількома вузлами і дата-центрами піднімає пропускну здатність на новий рівень.

Децентралізація (архітектура без лідера) означає, що кожен вузол може обробляти будь-який запит, читання або запис.

Лінійна масштабованість

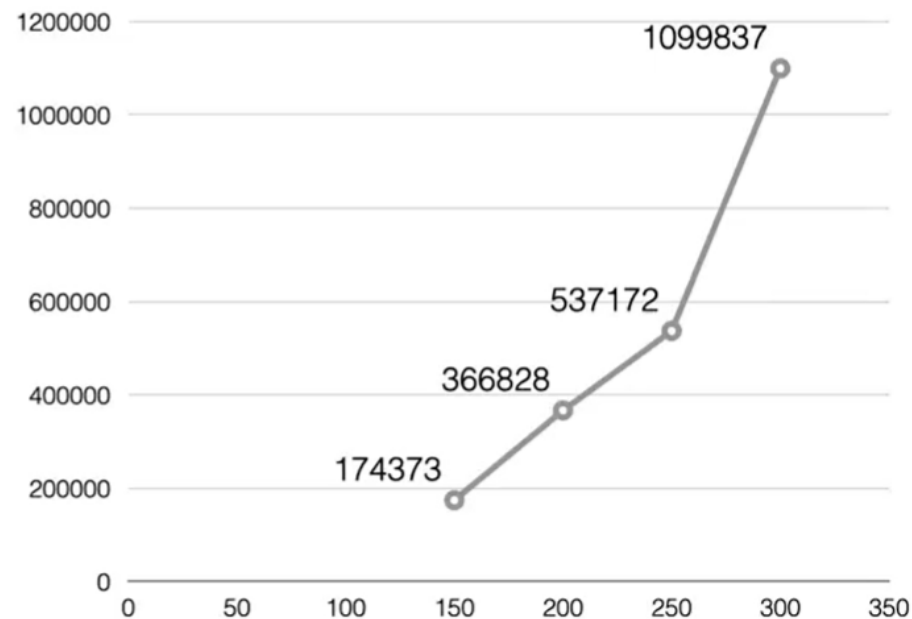
Linear Scalability

- For volume or velocity, there are no limitations
- Linear – no overhead on new nodes, scales with your needs

Немає обмежень на обсяг або швидкість і немає додаткових витрат на нові вузли.

Cassandra масштабується відповідно до ваших потреб.

Client Writes/s by node count - Replication Factor = 3

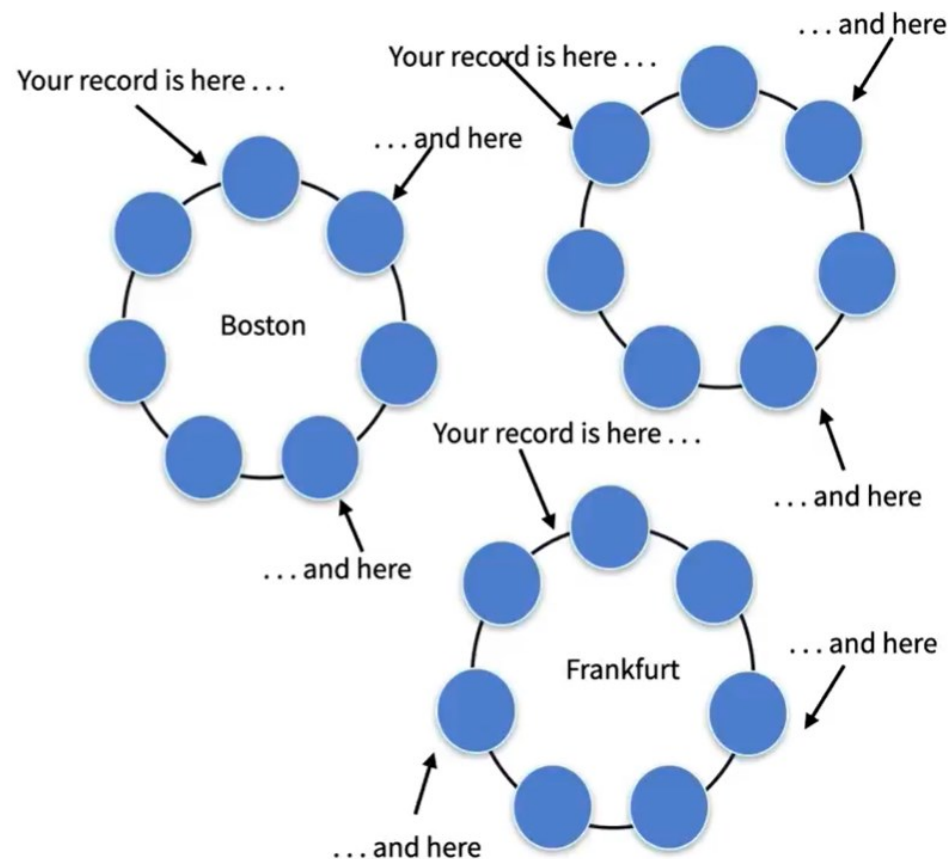


Найвища доступність

Highest Availability

- Replication takes care of possible downtimes
- Multiple live replicas
- No single point of failure
- Client-side smart reconnection and strong retry mechanism

Теоретично, ви можете досягти 100% часу безвідмовної роботи завдяки реплікації, децентралізації та стратегії розміщення, що враховує топологію.



Самовідновлення та автоматизація

Self-healing and automation:

- Operations for a huge cluster can be exhausting.
- Cassandra clusters alleviate a lot of headaches because they are smart — able to scale, change data replacement, and recover — all automatically.

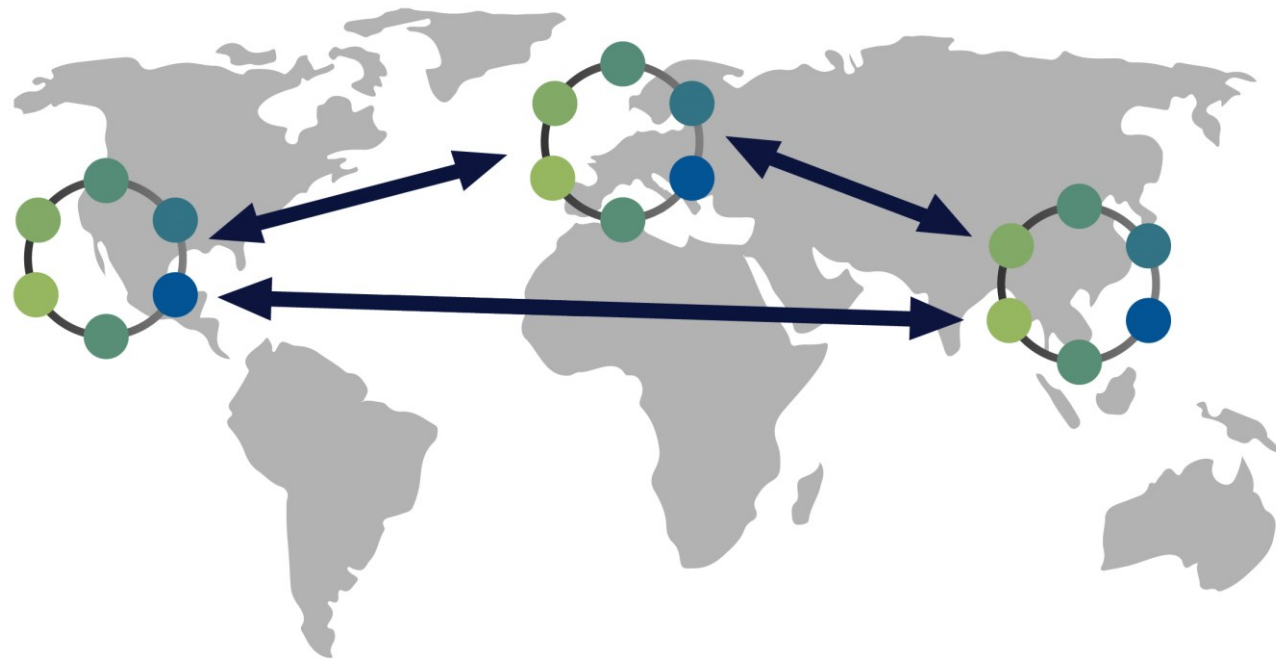
Операції для великого кластера можуть бути виснажливими.

Кластери Cassandra знімають багато головного болю, оскільки вони розумні — здатні масштабуватися, змінювати заміну даних і відновлюватися — все автоматично.

Географічний розподіл

Geographical Distribution

- Cassandra's trademark is multi-datacenter deployments, granting you an exceptional capacity for disaster tolerance while keeping your data close to your clients – worldwide.
- All datacenters are active (available for both writes and reads)!



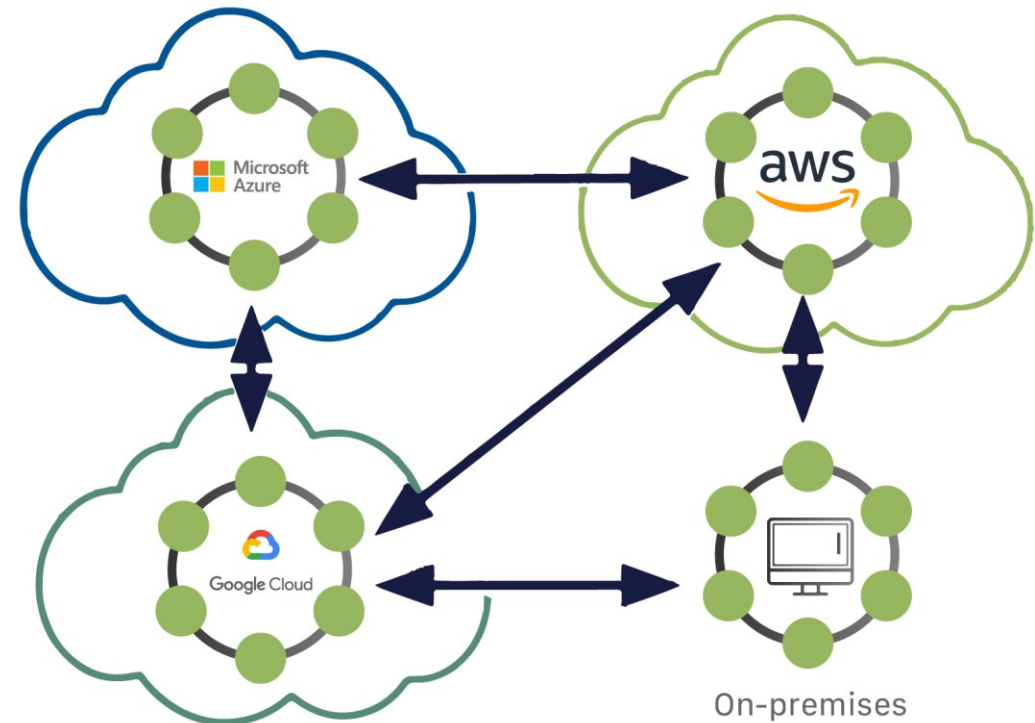
Розгортання в кількох дата-центрах надає виняткову здатність до стійкості до катастроф, зберігаючи ваші дані близько до клієнтів, де б вони не були у світі.

Незалежність від платформи

Platform Agnostic

- Apache Cassandra is not bound to any platform or service provider, helping you build hybrid cloud and multicloud solutions with ease

Cassandra не прив'язана до жодної платформи або постачальника послуг, що дозволяє легко створювати гібридні та мультихмарні рішення.



Незалежність від постачальників

Vendor Independent

- Cassandra doesn't belong to any commercial vendors
- It is controlled by the nonprofit The Apache Software Foundation
- Similar to Hadoop, Spark, Kafka, ZooKeeper, Maven, and many other projects.

Cassandra не належить жодному з комерційних постачальників, а пропонується некомерційною організацією Apache Software Foundation з відкритим кодом, що забезпечує як відкриту доступність, так і постійний розвиток.

Apache Cassandra

Cassandra може обробляти петабайти даних, і кілька вузлів можуть зберігати стільки даних, скільки потрібно. Cassandra також надзвичайно швидка, навіть для величезних наборів даних.

Cassandra підтримує горизонтальне масштабування, дозволяючи легко додавати нові вузли без зупинки системи.

Вона забезпечує високу доступність і відмовостійкість завдяки децентралізованій peer-to-peer архітектурі, де будь-який вузол може виконувати функції іншого.

Cassandra не має єдиної точки відмови, оскільки використовує реплікацію даних у кількох регіонах і дата-центрах.

Вона працює на будь-якій платформі, де підтримується Java Virtual Machine, і не прив'язана до конкретного постачальника послуг.

Це робить Cassandra ідеальним вибором для проектів з великими даними, які потребують високої продуктивності та надійності.

Архітектура: розподілена, безлідерна, peer-to-peer модель

Однією з важливих характеристик Cassandra є те, що її бази даних розподілені. Це забезпечує як технічні, так і бізнес-переваги.

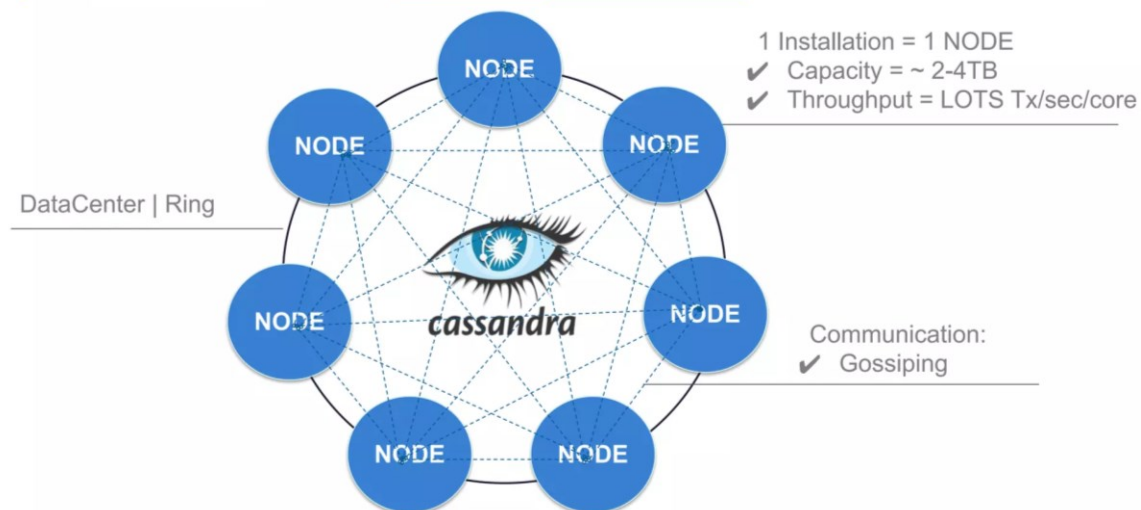
"Розподілена" означає, що Cassandra може працювати на кількох машинах, при цьому користувачі бачать її як єдине ціле.

Хоча можна запускати Cassandra на одному вузлі для ознайомлення, максимальну користь вона приносить під час роботи на кількох вузлах.

Вузли Cassandra спілкуються через протокол "gossip" і мають архітектуру без лідера, де кожен вузол може виконувати ті ж функції, що й інші. Вузли організовуються в кластери або "кільця", і можуть бути розгорнуті в кількох дата-центрах.

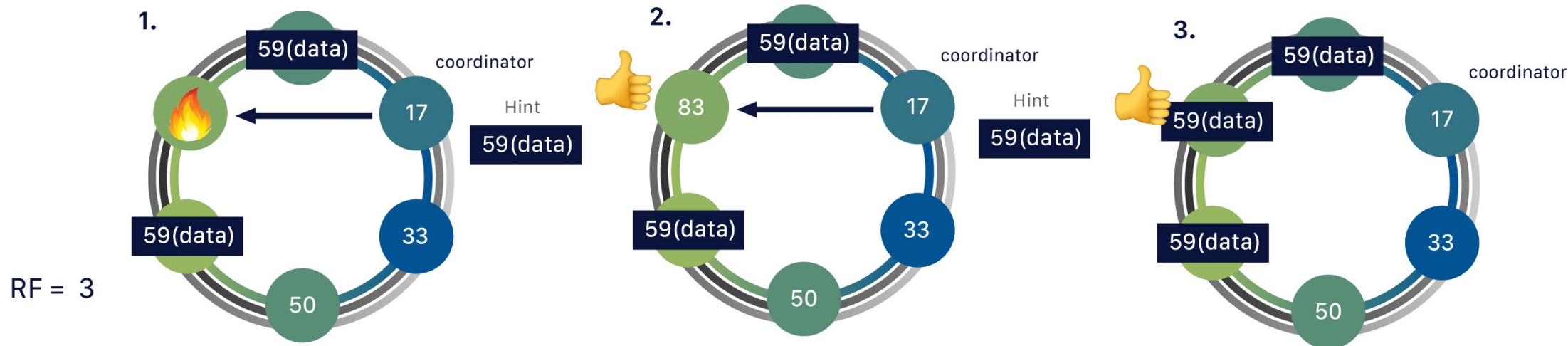
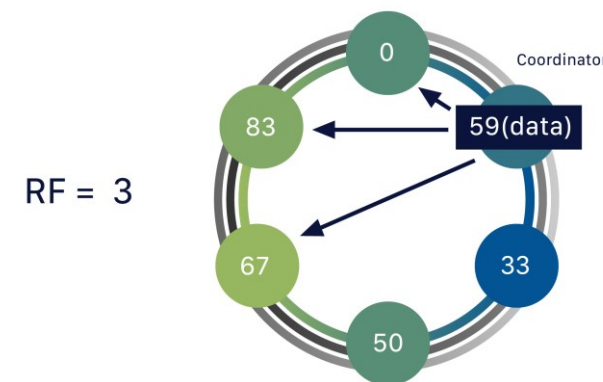
- **Розподілена система:**
Cassandra працює на кластері серверів, де кожен вузол має однакові права та обов'язки.
- **Безлідерна модель:**
Відсутність центрального вузла (лідера) означає, що всі вузли рівноправні, що підвищує стійкість до збоїв.
- **Peer-to-peer:**
Вузли взаємодіють один з одним безпосередньо, що дозволяє легко додавати нові вузли без зупинки системи.

Apache Cassandra™ = NoSQL Distributed Database



Особливості масштабування та реплікації

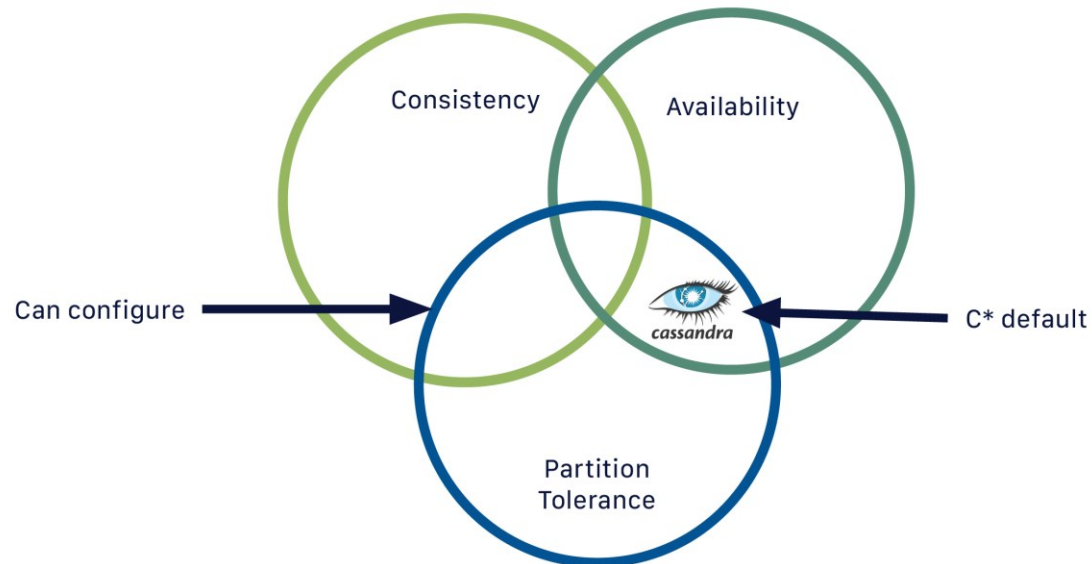
- **Горизонтальне масштабування:**
Легко додавайте нові вузли до кластера без зупинки системи.
- **Реплікація:**
Дані автоматично реплікуються на кілька вузлів для забезпечення відмовостійкості та доступності.



У Cassandra дані реплікуються. Фактор реплікації (RF) представляє кількість вузлів, які використовуються для зберігання ваших даних. Якщо $RF = 1$, кожен розділ зберігається на одному вузлі. Якщо $RF = 2$, то кожен розділ зберігається на двох вузлах і так далі. Промисловий стандарт — це фактор реплікації три, хоча є випадки, коли потрібно використовувати більше або менше вузлів.

Консистентність, доступність та розподіл даних (CAP теорема)

- **CAP теорема:**
Cassandra обирає доступність (Availability) та стійкість до розділення (Partition Tolerance) над жорсткою консистентністю (Consistency).
- **Тюнінг консистентності:**
Користувачі можуть налаштовувати рівень консистентності для читання та запису, вибираючи між швидкістю та точністю даних.

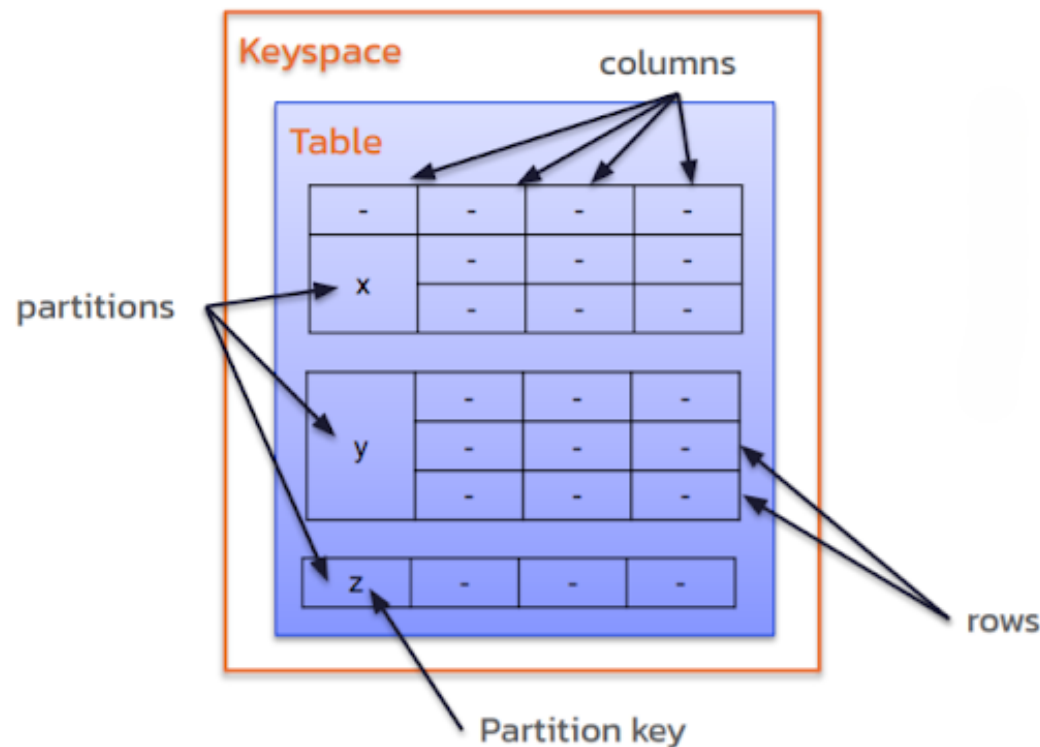


Розділення даних

Cassandra використовує **розподіл на основі ключів** для розподілу даних.

Основні компоненти структури даних Cassandra включають:

- **Keyspace (ключовий простір):**
Контейнер для даних, схожий на схему, який містить кілька таблиць.
- **Table (Таблиця):**
Набір стовпців, первинний ключ і рядки, що зберігають дані в розділах.
- **Partition (Розділ):**
Група рядків з однаковим токеном розділу, що є базовою одиницею доступу в Cassandra.
- **Row (Рядок):**
Окремий, структурований елемент даних у таблиці.

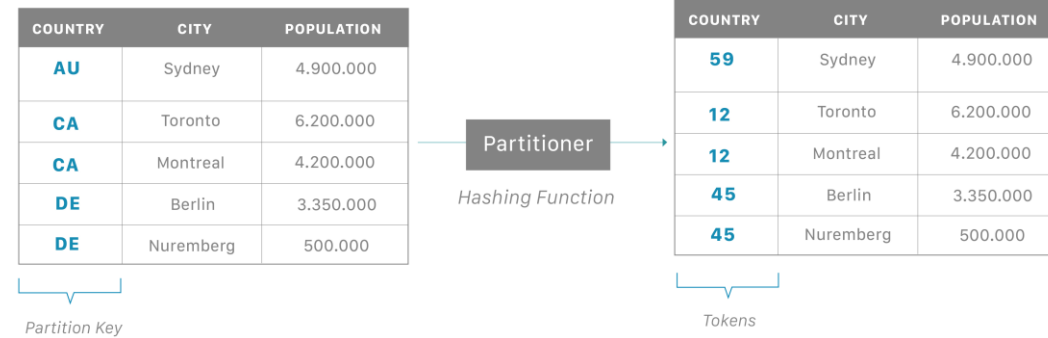


*Розділ — це група рядків з однаковим токеном розділу або значенням для кожного елемента в ключі розділу. Зберігаючись разом, всі рядки, що мають спільний токен розділу, будуть сусідами.
Таблиця — це група стовпців і рядків, що зберігають дані на основі ключа розділу для цієї таблиці.*

Розділення даних

Cassandra зберігає дані в розділах, які представляють собою набір рядків у таблиці по всьому кластеру.

Кожен рядок містить ключ розділу — один або кілька стовпців, які хешуються для визначення того, як дані розподіляються по вузлах у кластері (перетворення в токени).



COUNTRY	CITY	POPULATION
USA	New York	8.000.000
USA	Los Angeles	4.000.000
FR	Paris	2.230.000
DE	Berlin	3.350.000
UK	London	9.200.000
AU	Sydney	4.900.000
DE	Nuremberg	500.000
CA	Toronto	6.200.000
CA	Montreal	4.200.000
FR	Toulouse	1.100.000
JP	Tokyo	37.430.000
IN	Mumbai	20.200.000

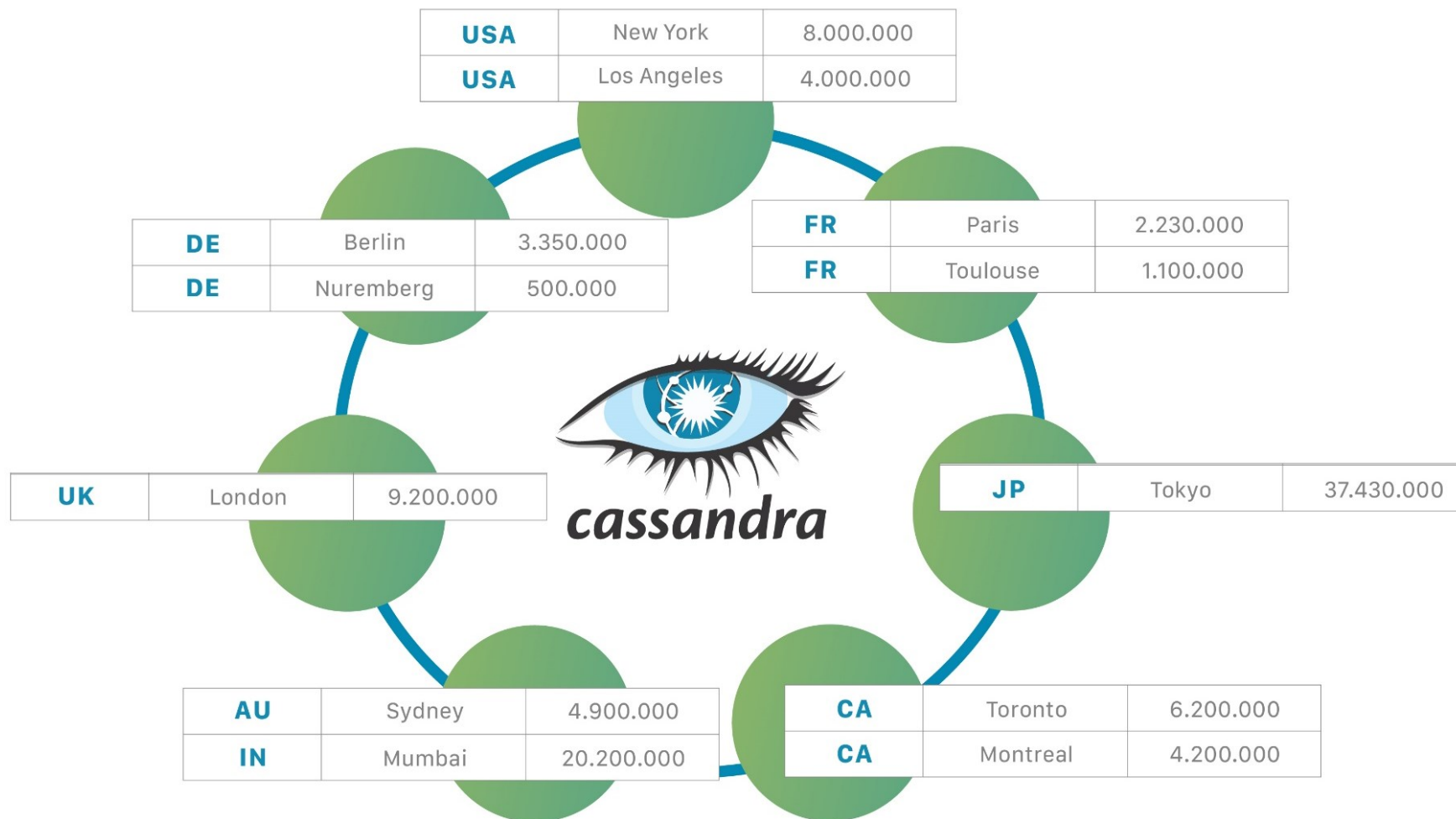
Partition Key

У нас є три стовпці: "Country", "City" і "Population".

У цьому випадку "Country" є нашим ключем розділу. Тобто ми розділяємо наші рядки на основі значення "Country".

Оскільки елементи з кожною країною мають однаковий ключ розділу, вони будуть зберігатися разом у вузлі, і дані автоматично розподілятимуться по вузлах кластера.

Розділення даних



Елементи, що мають спільні ключі розділу, зберігаються разом, тому запити до цих даних будуть надзвичайно швидкими.

Чому розподіл?

Розподіл значно полегшує масштабування. Великі дані не можуть вміститися на одному сервері, тому вони розбиваються на частини, які можна легко розподілити по десятках, сотнях або навіть тисячах серверів, додаючи більше за потреби.

Після того, як ви встановите ключ розділу для своєї таблиці, розподільник перетворює значення в ключі розділу на токени (також звані хешуванням) і призначає кожному вузлу діапазон даних, званий діапазоном токенів.

Cassandra потім автоматично розподіляє кожен рядок даних по кластеру за значенням токена. Якщо вам потрібно масштабуватися, просто додайте новий вузол, і ваші дані будуть перерозподілені відповідно до нових призначень діапазону токенів. З іншого боку, ви також можете зменшити масштаб без проблем.

Переваги та недоліки використання Cassandra

Переваги:

- Висока доступність та відмовостійкість.
- Масштабованість без зупинки системи.
- Гнучкість у налаштуванні консистентності.

Недоліки:

- Складність у налаштуванні та адмініструванні.
- Відсутність транзакційної підтримки, як у реляційних базах даних.
- Можливі затримки при досягненні консистентності в розподілених системах.

Мова запитів CQL (Cassandra Query Language)

Cassandra Query Language (CQL) — це мова запитів, розроблена для роботи з базою даних Apache Cassandra. Вона подібна на SQL, але адаптована для роботи з колонковими структурами даних, які використовує Cassandra.

Підтримує основні операції, такі як створення таблиць, вставка, оновлення та видалення даних.

Мова запитів CQL (Cassandra Query Language)

Створення Keyspace

```
CREATE KEYSPACE my_keyspace WITH replication = {  
    'class': 'SimpleStrategy',  
    'replication_factor': 3  
};
```

Цей запит створює новий keyspace з назвою **my_keyspace** з простою стратегією реплікації і фактором реплікації 3.

Створення таблиці

```
CREATE TABLE my_keyspace.users (  
    user_id UUID PRIMARY KEY,  
    first_name TEXT,  
    last_name TEXT,  
    email TEXT  
);
```

Цей запит створює таблицю **users** в keyspace **my_keyspace** з колонками **user_id**, **first_name**, **last_name** і **email**.

Мова запитів CQL (Cassandra Query Language)

Вставка даних

```
INSERT INTO my_keyspace.users (user_id, first_name, last_name, email)  
VALUES (uuid(), 'John', 'Doe', 'john.doe@example.com');
```

Цей запит вставляє новий рядок у таблицю **users** з вказаними значеннями.

Вибірка даних

```
SELECT first_name, last_name  
FROM my_keyspace.users  
WHERE user_id = some_uuid;
```

Цей запит вибирає **first_name** і **last_name** з таблиці **users** для конкретного **user_id**.

Мова запитів CQL (Cassandra Query Language)

Оновлення даних

```
UPDATE my_keyspace.users  
SET email = 'new.email@example.com'  
WHERE user_id = some_uuid;
```

Цей запит оновлює адресу електронної пошти для користувача з вказаним **user_id**.

Видалення даних

```
DELETE FROM my_keyspace.users  
WHERE user_id = some_uuid;
```

Цей запит видаляє рядок з таблиці **users** для конкретного **user_id**.

Видалення таблиці

```
DROP TABLE my_keyspace.users;
```

Цей запит видаляє таблицю **users** з keyspace **my_keyspace**.



ДЯКУЮ ЗА УВАГУ!